

10-Day Intern Technical Training & Domain Assessment Program

Self-Study • Hands-on Practice • Technical Assessment

Program Overview

Item	Details
Duration	10 Working Days
Mode	Self-Study + Hands-on Development
Initial Domains	AI/ML and Web Development
Web Stack	React, Next.js, Node.js, Laravel, TypeScript, Angular
Future Mobile Path	Selected Web Development interns may move to Ionic/Angular Mobile Development

Program Objective

During these 10 days, you will independently study the provided learning material, complete exercises, build practical assignments, and work on a final project. The objective is to develop practical technical skills and demonstrate your ability to learn, solve problems, build software, debug issues, and explain your work.

- Study the assigned material independently.
- Complete all exercises and practical assignments.
- Maintain your work using Git.
- Document your implementation and decisions.
- Research and troubleshoot problems independently before asking for help.
- Be prepared to explain the code and technical decisions in your submissions.

10-Day Roadmap

Day	Area	Main Topics	Practical Work
1	Programming Fundamentals	Programming, Git, problem solving	Programming challenge
2	Python	Python fundamentals, OOP, files	Python application
3	Data & AI/ML	NumPy, Pandas, visualization	Data analysis project
4	Machine Learning	ML concepts, algorithms, evaluation	ML prediction project
5	JavaScript	Modern JS, async, APIs	JavaScript application
6	TypeScript + React	TypeScript, React, hooks	React application
7	Next.js + Backend	Next.js, Node.js, REST APIs	Full-stack application
8	Database + Laravel	SQL, database, Laravel	CRUD API
9	Angular	Angular, RxJS, services, APIs	Angular application
10	Final Project	Full-stack / AI/ML	Project + presentation

DAY 1 — Programming Fundamentals & Problem Solving

Objective: Establish strong programming fundamentals and problem-solving ability.

Programming Basics

- Variables and constants
- Data types and operators
- Conditional statements
- Loops
- Functions
- Arrays, strings and objects
- Scope
- Error handling
- Basic OOP

Data Structures & Algorithms

- Arrays
- Stack
- Queue
- Hash Map / Dictionary
- Set
- Searching and sorting
- Recursion
- Time and space complexity

Git & GitHub

- Repository, commit, branch, merge, pull, push and pull requests

Exercises / Practice

Solve at least 15 programming problems. Examples: reverse a string; palindrome; largest/second-largest number; remove duplicates; missing/duplicate number; character frequency; first non-repeating character; merge sorted arrays; common elements; stack; queue; maximum subarray sum; sorting without built-in sorting.

Practical Assignment

Build an Employee Management CLI application with Add, Update, Delete, Search, List, Highest Salary, Average Salary and Department Filter features.

Deliverables

```
day-01/  
├── exercises/  
├── employee-management/  
└── README.md
```

DAY 2 — Python Development

Objective: Learn Python well enough to write maintainable programs and process data.

Python

- Syntax
- Variables and data types
- Lists, tuples, sets and dictionaries
- Conditions and loops
- Functions
- Lambda and list comprehensions
- Modules and packages
- Exception handling

- File handling
- Classes, objects, inheritance and encapsulation
- Virtual environments and pip

Exercises / Practice

Implement a student/employee management system. Build a CSV/JSON reader that reports record count, missing values, duplicates, averages, minimum/maximum values and category-wise statistics.

Practical Assignment

Build a Python management system supporting Add, Update, Delete, Search, Filter, Sort and Statistics. Store data in JSON or CSV and implement exception handling.

Deliverables

```
day-02/
├── python-exercises/
├── management-system/
├── csv-analysis/
└── README.md
```

DAY 3 — Data Analysis & Python for AI/ML

Objective: Learn to inspect, clean, analyze and visualize real-world data.

NumPy

- Arrays
- Dimensions and shape
- Indexing and slicing
- Mathematical and aggregation operations

Pandas

- Series and DataFrames
- Reading CSV
- Filtering
- Sorting
- Grouping and aggregation
- Missing values
- Duplicates
- Data transformation

Visualization

- Matplotlib
- Bar charts
- Line charts
- Histograms
- Scatter plots

Exercises / Practice

Analyze a facility dataset containing fields such as facility_id, location, cleanliness_score, odor_score, waste_level, water_availability, footfall, complaints and inspection_date. Identify missing/duplicate/invalid data and outliers. Calculate key statistics and identify at least three useful insights. Create 2 bar charts, 1 histogram, 1 scatter plot and 1 additional visualization.

Practical Assignment

Produce a cleaned dataset, analysis notebook/scripts, visualizations and a README explaining findings.

Deliverables

```
day-03/
├── dataset/
├── data-cleaning/
├── analysis/
├── visualizations/
└── README.md
```

DAY 4 — Machine Learning

Objective: Understand core ML concepts and build a basic prediction model.

ML Fundamentals

- AI, ML and Deep Learning
- Supervised and unsupervised learning
- Classification and regression
- Features and labels
- Training, validation and testing

ML Concepts

- Overfitting
- Underfitting
- Bias and variance
- Data leakage
- Feature engineering
- Feature selection

Algorithms

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- K-Means

Evaluation

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- MAE
- MSE
- RMSE

Exercises / Practice

Build a model that predicts facility hygiene risk using cleanliness_score, odor_score, waste_level, complaints, footfall and hours_since_cleaning. Use the workflow: dataset → cleaning → EDA → feature engineering → train/test split → model training → prediction → evaluation. Compare at least two algorithms.

Practical Assignment

Document the selected model, features, performance, problems encountered and possible improvements.

Deliverables

```
day-04/  
├── dataset/  
├── preprocessing/  
├── models/  
├── evaluation/  
├── predictions/  
└── README.md
```

DAY 5 — JavaScript

Objective: Learn modern JavaScript and understand asynchronous programming and APIs.

JavaScript Fundamentals

- let/const
- Data types
- Functions and arrow functions
- Arrays and objects
- Destructuring
- Spread/rest
- Template literals

Array Methods

- map
- filter
- reduce
- find
- some
- every
- sort
-

Advanced JavaScript

- Closures
- Scope
- Hoisting
- Callbacks
- Promises
- async/await
- Event loop
- Modules
- Error handling

APIs

- HTTP
- JSON
- Fetch
- REST APIs
- GET, POST, PUT and DELETE

Exercises / Practice

Build a JavaScript Employee Dashboard that consumes API/JSON data and supports listing, search, filter, sort, details, add, edit and delete operations.

Practical Assignment

Use modern JavaScript and reusable functions. Avoid unnecessary libraries.

Deliverables

```
day-05/  
├── javascript/  
├── employee-dashboard/  
└── README.md
```

DAY 6 — TypeScript + React

Objective: Learn TypeScript and build a component-based React application.

TypeScript

- Primitive types
- Arrays and objects
- Interfaces
- Type aliases
- Union types
- Enums
- Optional properties
- Functions
- Classes
- Generics
- Type narrowing
- Type guards

React

- Components
- JSX
- Props
- State
- Events
- Forms
- Conditional rendering
- Lists
- Component composition
- useState
- useEffect
- Custom hooks
- API calls
- Error handling

Exercises / Practice

Convert the Day 5 employee application to TypeScript and build a React Employee Management Dashboard.

Practical Assignment

Dashboard should display total employees, average salary and departments. Employee features should include list, search, filter, sort, details, add, edit and delete. Use reusable components, validation and API integration.

Deliverables

```
day-06/
├── typescript/
├── react-app/
└── README.md
```

DAY 7 — Next.js + Node.js

Objective: Understand modern React application development and backend API development.

Next.js

- App Router
- Pages and layouts
- Dynamic routes
- Server and client components
- Data fetching
- Loading and error states
- API routes
- Environment variables

Node.js + Express

- Node.js and npm
- Express
- Routing
- Controllers
- Middleware
- REST APIs
- Request/response
- Validation
- Authentication basics
- Error handling

Exercises / Practice

Convert the Employee Dashboard into a Next.js application. Create `/employees`, `/employees/[id]` and `/employees/create`. Build a Node.js/Express API with GET, POST, PUT and DELETE employee endpoints.

Practical Assignment

Use a logical backend structure: routes, controllers, services, models, middleware and utils.

Deliverables

```
day-07/
├── nextjs-app/
├── node-api/
└── README.md
```

DAY 8 — Database + Laravel

Objective: Understand relational databases and implement backend APIs using Laravel.

SQL & Databases

- Tables, rows and columns
- Primary and foreign keys
- Relationships
- CRUD
- SELECT, WHERE, ORDER BY, GROUP BY, HAVING
- JOIN
- Subqueries
- Indexes
- Transactions

Laravel

- Architecture
- Routing
- Controllers
- Models
- Migrations
- Eloquent ORM
- Relationships
- Validation
- Middleware
- Authentication basics
- API development
- CRUD

Exercises / Practice

Create users, departments, employees, facilities, inspections and complaints tables with appropriate relationships. Write SQL queries for department employees, average salary, highest-paid employee, poor facilities, complaint counts and inspection history. Build a Laravel CRUD API for Facilities, Inspections and Complaints.

Practical Assignment

Understand the flow: Request → Route → Controller → Model → Database → Response.

Deliverables

```
day-08/  
├── sql/  
├── database/  
├── laravel-api/  
└── README.md
```

DAY 9 — Angular + TypeScript + API Integration

Objective: Build an Angular application and demonstrate skills relevant to future Ionic mobile development.

Angular

- Architecture
- Components
- Templates
- Data binding
- Property/event/two-way binding
- Directives
- Pipes

- Services
- Dependency Injection
- Routing
- Route parameters
- Forms and Reactive Forms
- HTTP Client
- Observables
- RxJS basics
- API integration
- Error handling

Exercises / Practice

Build a Facility Inspection Dashboard with dashboard metrics, facility list/search/filter/sort/details, inspection form and inspection history. Connect the application to a REST API.

Practical Assignment

Use TypeScript interfaces/models for API data and organize the application into components, pages, services, models and guards where appropriate.

Deliverables

```
day-09/
├── angular-app/
├── api-integration/
└── README.md
```

DAY 10 — Final Project

Objective: Demonstrate your ability to independently design, develop, debug, document and explain a working system.

Exercises / Practice

Choose one primary implementation: AI/ML or Web Development.

Practical Assignment

AI/ML: Build a Smart Hygiene Risk Prediction System with preprocessing, EDA, feature selection, model training/evaluation, prediction and visualization. Bonus: expose predictions through an API.

Web: Build a Smart Facility Management Dashboard using React/Next.js, Node.js/Laravel and PostgreSQL/MySQL. Include dashboard, facilities, inspections and complaints. Include at least one functional Angular module/page connected to an API.

Deliverables

```
day-10/
├── final-project/
│   ├── frontend/
│   ├── backend/
│   ├── database/
│   ├── ml/
│   └── README.md
```

Final Project — Common Requirements

- Responsive and usable interface where applicable.
- Reusable components and functions.

- Form validation and error handling.
- REST API integration where applicable.
- Proper database schema and relationships where applicable.
- Meaningful variable, function and component names.
- Clean project structure and minimal code duplication.
- Testing of the main application flows.
- Proper Git history and documentation.

README Requirements

Every major assignment and the final project must contain a README with:

- Project Overview
- Problem Statement
- Features
- Technology Stack
- Architecture
- Database Design, where applicable
- API Documentation, where applicable
- Installation
- Environment Variables
- How to Run
- Screenshots, where applicable
- Challenges Faced
- Solutions
- Future Improvements

Daily Submission Checklist

- Assigned learning material completed.
- Exercises completed.
- Practical assignment completed.
- Code committed to Git.
- README updated.
- Application/code tested.
- Problems encountered and solutions documented.

Before Asking for Help

Understand the problem

↓

Read the error

↓

Check documentation

↓

Search for possible solutions

↓

Try a solution

↓

Debug

↓

Ask for help if still blocked

You should be able to explain every important part of the code you submit. Using documentation, search engines and AI tools is permitted where applicable, but you are responsible for understanding and validating the final implementation.

Final Presentation

Each intern will present their final project in approximately 10–15 minutes.

1. Explain the problem you solved.
2. Explain the technology stack.
3. Explain the architecture and data flow.
4. Demonstrate the working application/model.
5. Explain the most difficult problem you encountered and how you solved it.
6. Explain important technical decisions.
7. Explain what you would improve with another week of development.

Potential Development Paths After the Program

Path	Expected Progression
AI/ML	Python → Data Analysis → Machine Learning → Deep Learning → AI Applications
Web Development	JavaScript → TypeScript → React/Next.js → Node.js/Laravel → Database → Angular
Web → Mobile	TypeScript → Angular → Ionic → Mobile Application Development

Final Submission Structure

```
intern-week-1/  
├── day-01/  
├── day-02/  
├── day-03/  
├── day-04/  
├── day-05/  
├── day-06/  
├── day-07/  
├── day-08/  
├── day-09/  
└── day-10/  
    └── final-project/
```

Study Materials — Documentation & Video Resources

The following free, official documentation and video resources correspond to each day of the roadmap. Use the documentation as your primary reference and the video course to build hands-on understanding before starting each day's exercises and practical assignment.

Day 1 — Programming Fundamentals, DSA & Git

Documentation:

- [MDN JavaScript Guide](#)
- [Git Documentation](#)
- [GitHub Docs](#)

Video Course:

- [Git & GitHub Crash Course for Beginners — freeCodeCamp](#)

Day 2 — Python Development

Documentation:

- [Python 3 Official Documentation](#)
- [Python Official Tutorial](#)

Video Course:

- [Learn Python — Full Course for Beginners — freeCodeCamp](#)

Day 3 — NumPy, Pandas & Visualization

Documentation:

- [NumPy Documentation](#)
- [Pandas Documentation](#)
- [Matplotlib Documentation](#)

Video Course:

- [Learn Python for Data Science — Full Course \(NumPy & Pandas\) — freeCodeCamp](#)

Day 4 — Machine Learning

Documentation:

- [scikit-learn User Guide](#)
- [scikit-learn API Reference](#)

Video Course:

- [Machine Learning for Everybody — Full Course — freeCodeCamp](#)

Day 5 — JavaScript

Documentation:

- [MDN JavaScript Reference](#)
- [MDN Asynchronous JavaScript Guide](#)

Video Course:

- [Learn JavaScript — Full Course for Beginners — freeCodeCamp](#)

Day 6 — TypeScript + React

Documentation:

- [TypeScript Handbook](#)
- [React Documentation](#)

Video Course:

- [Learn React — Full Course for Beginners — freeCodeCamp](#)

Day 7 — Next.js + Node.js

Documentation:

- [Next.js Documentation](#)
- [Express.js Documentation](#)

Video Course:

- [Learn Node.js and Express — Full Course — freeCodeCamp](#)
- [Learn Next.js — Full Course — freeCodeCamp](#)

Day 8 — Database + Laravel

Documentation:

- [Laravel Documentation](#)
- [MySQL Documentation](#)

Video Course:

- [Learn Laravel by Building a Medium Clone — freeCodeCamp](#)

Day 9 — Angular

Documentation:

- [Angular Documentation](#)
- [RxJS Documentation](#)

Video Course:

- [Angular for Beginners Course + TypeScript — freeCodeCamp](#)

Day 10 — Final Project

Documentation:

- [freeCodeCamp — Full Curriculum & Practice](#)

Video Course:

- Revisit the Day 4 and Day 6–9 videos above depending on the chosen path (AI/ML or Web)